부록 · Spring Boot — Java · Kotlin

Spring Boot REST API 정석 입문

DTO + JPA + Entity + Repository : Java와 Kotlin 쉬운 비교

1. REST API 기본 흐름

JSON → Controller → DTO → Service → Entity → Repository → JPA → MariaDB

2. 핵심 개념 비교

구분	쉬운 뜻	대표 예
DTO	데이터를 주고받는 상자	MemberDto
Entity	DB 테이블을 객체로 표현	Member
Repository	DB 저장·조회·수정·삭제 담당	MemberRepository
JPA	객체와 DB의 영속성을 연결·관리하는 표준 기술	save(), findAll() 등의 기반

3. Java + Spring Boot

Java DTO

```java
public class MemberDto {
    private String name;
    private String email;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Java Entity

```java
@Entity
@Table(name = "member")
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Java Repository

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
}
```

Java Service

```java
@Service
public class MemberService {
    private final MemberRepository repository;

    public MemberService(MemberRepository repository) {
        this.repository = repository;
    }

    public Member save(MemberDto dto) {
        Member member = new Member();
        member.setName(dto.getName());
        member.setEmail(dto.getEmail());
        return repository.save(member);
    }

    public List<Member> findAll() {
        return repository.findAll();
    }
}
```

Java Controller

```java
@RestController
@RequestMapping("/members")
public class MemberController {
    private final MemberService service;

    public MemberController(MemberService service) {
        this.service = service;
    }

    @PostMapping
    public Member save(@RequestBody MemberDto dto) {
        return service.save(dto);
    }

    @GetMapping
    public List<Member> findAll() {
        return service.findAll();
    }
}
```

4. Kotlin + Spring Boot

Kotlin DTO

```kotlin
data class MemberDto(
    val name: String,
    val email: String
)
```

Kotlin Entity

```kotlin
@Entity
@Table(name = "member")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String = ""
)
```

Kotlin Repository

```kotlin
interface MemberRepository :
    JpaRepository<Member, Long>
```

Kotlin Service

```kotlin
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun save(dto: MemberDto): Member {
        val member = Member(
            name = dto.name,
            email = dto.email
        )
        return repository.save(member)
    }

    fun findAll(): List<Member> =
        repository.findAll()
}
```

Kotlin Controller

```kotlin
@RestController
@RequestMapping("/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun save(@RequestBody dto: MemberDto): Member =
        service.save(dto)

    @GetMapping
    fun findAll(): List<Member> =
        service.findAll()
}
```

5. Java와 Kotlin 비교

항목	Java + Spring Boot	Kotlin + Spring Boot
DTO	class + getter/setter	data class
Entity	@Entity class	@Entity class
Repository	extends JpaRepository	: JpaRepository
Service	@Service	@Service
Controller	@RestController	@RestController
JPA 저장	repository.save()	repository.save()
JPA 조회	repository.findAll()	repository.findAll()
구조	동일	동일
코드량	비교적 많음	간결함

6. REST API 동작 예

POST /members 요청 JSON

```json
{
  "name": "홍길동",
  "email": "hong@test.com"
}
```

POST /members → Controller → DTO → Service → Entity → Repository → JPA → MariaDB

7. REST API 정석 구조

```text
dto/
 ├─ MemberRequestDto
 └─ MemberResponseDto

entity/
 └─ Member

repository/
 └─ MemberRepository

service/
 └─ MemberService

controller/
 └─ MemberController
```

학습 순서: POST 등록 → GET 전체조회 → GET 한 건 조회 → PUT 수정 → DELETE 삭제

8. 핵심 암기

DTO = API 데이터를 주고받는 상자

Entity = DB 테이블을 객체로 표현

Repository = Entity의 CRUD 담당

JPA = 객체와 DB의 영속성을 연결·관리하는 표준

정석 흐름 = DTO → Controller → Service → Entity → Repository → JPA → MariaDB
